Datatypes → Primitive → Num, Strings, Boolean, Null, Undefined
→ Non-Primitive → objects, Arrays, functions.

Operators

It is a keyword which is used for performing an operation on values, such as calculation, comparisions and more.

(i) Arithmetic Operators-

- Addition (+), • Substraction (-), • Multiplication (*)
- Division (/), • Modulus (%), • Exponentation (**)

(ii) Assignment Operators -

- Assignment (=), • Addition assigment (+=)
- Substract assign. (-=), • Multiply assign. (*=)
- Divison assign (/=)

(iii) Comparison Operators -

- equal to (==), • not equal to (!=)
- greater than (>), • greater than (<)
- greater than or equals to (>=)
- ~~greater~~ smaller than or equals to (<=)

iv) Logical Operators -

Logical AND (&&) → both true
Logical OR (||) → One true
Logical NOT (!) → Inverts.

v) Conditional Operator – provides a short syntax for conditionally assigning a value based on condition.

Syntax:

* condition ? if true : if false ;

Control Statements

Control statements in JS are used to control the flow of execution of the program - i.e to decide which part of the code should run, how many times & under what condition.

↳ 1. Conditional Statements
↳ 2. Looping (Iterative) Statements
↳ 3. Jump (Branching) Statements.

1. Conditional Statements - these statements are used to make decisions based on conditions (true/ false)

↳ If & If-else statements & If else-If state.

Syntax -

```
If (condition) {
....
}
```

Syntax.

```
if (condition) {
....
} else {
....
}
```

↳ Switch statement → used when there are many possible values for a variable It an alternative to multiple If-else statements

Syntax -

```
switch (expression) {
case value1:
    // code
    break;
case value2:
    // code
    break;
case value3:
    // code
    break;
default //code
}
```

2. Looping (Iterative) Statement →

Used to execute a block of code repeatedly

↳ for Loop → runs a block of code a specific numbers of times.

Syntax -

```
for (initialization ; condition ; inc/dec) {
    // block of code
}
```

↳ while loop → executes a block while the condition is true.

Syntax

```
while (condition) {
    // code
}
```

↳ do while → Its runs atleast once.

```
do {
    // Code
} while (condition);
```

3. Jump Statements → used to change the normal sequence of execution.

↳ break statement → used to exit from a loop.

Ex-

```
for (let i=1 ; i<=10 ; i++) {
    if (i==5) break;
    console.log (i);
}                          // output = 1 2 3 4
```

↳ Continue Statement → used to skip the current iteration

Ex-

```
for (let i=1 ; i<=5 ; i++) {
    if (i==3)
    console.log (i);
}                          // output : 1 2 4 5
```

↳ Return Statement → used to exit from a function.

Ex-

```
function add (a, b) {
    return a+b;
}

console.log (add (5, 3); // 8
```